

QMST5336: ANALYTICS

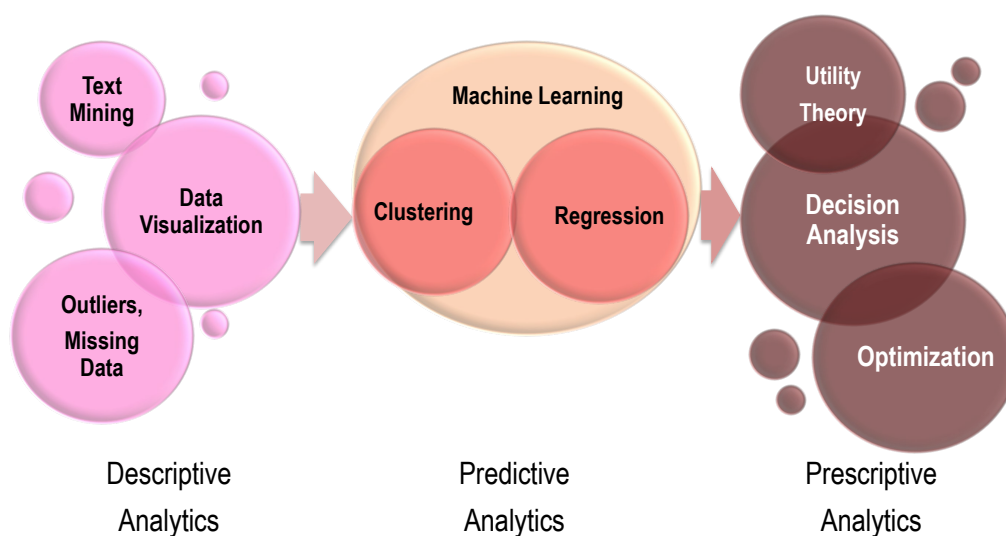
Lecture 7: Regression II Time Series

Emily Zhu, PhD.

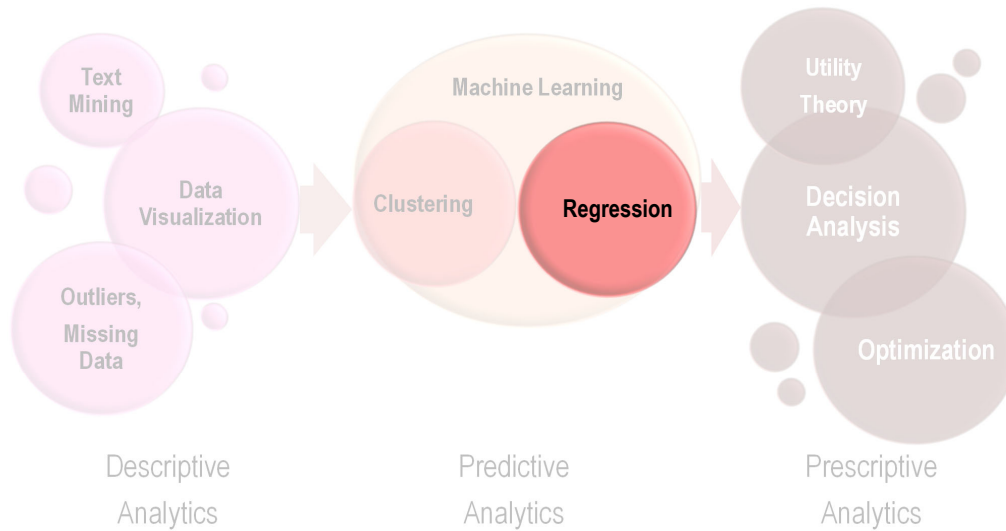
© 2023 Texas State University

Our Problem and Roadmap

An investment property in Austin: **Airbnb** versus **Rental**?

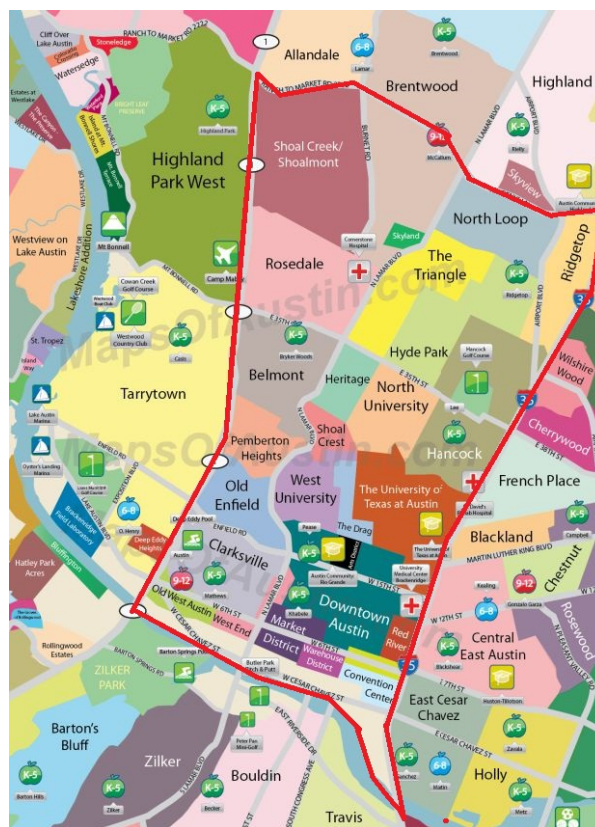


Roadmap: Where We Are



Case: How to Forecast Airbnb Demand?

Airbnb hosts are always interested in forecasting demand in their local market.



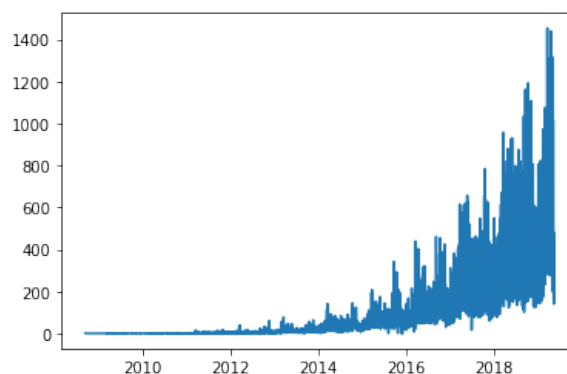
Review Frequencies

	id	zipcode	room_type	bed_type	accommodates	bathrooms	bedrooms	beds	reviews_per_month	security_deposit	cleaning_fee	price	property_type
0	2265	78702.0	Entire home/apt	Real Bed	4	2.0	2.0	2.0	0.19	500.0	100.0	225.0	House
1	5245	78702.0	Private room	Real Bed	2	1.0	1.0	1.0	0.07	500.0	75.0	100.0	House
4	25028	78757.0	Entire home/apt	Real Bed	4	1.0	2.0	2.0	0.28	0.0	100.0	100.0	House
5	39993	78703.0	Entire home/apt	Real Bed	2	1.0	1.0	1.0	1.75	0.0	20.0	59.0	House
6	46929	78703.0	Entire home/apt	Real Bed	4	1.0	1.0	2.0	3.30	0.0	40.0	105.0	House

	listing_id	id	date	reviewer_id	reviewer_name	comments
0	1078	142	2008-09-13	1344	Amy	A fabulously cozy place! It was slightly diff...
1	1078	207074	2011-03-22	451657	Angela	perfect!---;) Will stay again!
2	1078	217779	2011-04-04	473682	Jenn	I had a great stay at Brian's garden apartment...
3	1078	6841593	2013-08-26	8153398	Kathryn	The reservation was canceled 19 days before ar...
4	1078	7071134	2013-09-05	420381	Rita	Fantastic Experience! My daughter and I staye...

Case: Data Pre-Processing

```
1 import numpy as np
2 dd_start = np.datetime64(min(df_full.date.unique()), 'D')
3 dd_start
4
5 numpy.datetime64('2008-09-13')
6
7 from matplotlib import pyplot as plt
8 n, x, p = plt.hist(num_days, bins = 3895)
9 a_days = np.arange('2008-09-13', '2019-05-14', dtype='datetime64[D]')
10 plt.plot(a_days, n)
```



Case: Data Pre-Processing C'td

```
1 #convert date column into datetime object
2 df1['index'] = df1['index'].astype('datetime64[ns]')
3 #Resample to weekly data
4 df2 = df1.resample('W', label='right', closed = 'right', on='index'
5     ).mean()#sum()
6 df_t = df2.loc['2012-06-01':]
7 df_w.n_reviews = np.log(df_t.n_reviews + 1)
```

n_reviews	
index	
2012-06-03	1.665008
2012-06-10	1.386294
2012-06-17	1.455287
2012-06-24	1.386294
2012-07-01	1.488077

Time Series: Introduction

Forecasting methods can be classified as qualitative or quantitative.

Qualitative methods generally involve the use of expert judgment to develop forecasts.

Quantitative forecasting methods can be used when:

- Past information about the variable being forecast is available.
- The information can be quantified.
- It is reasonable to assume that past is prologue.

The objective of time series analysis is to uncover a pattern in the time series and then extrapolate the pattern into the future.

The forecast is based solely on past values of the variable and/or on past forecast errors.

Modern data-collection technologies have enabled individuals, businesses, and government agencies to collect vast amounts of data that may be used for forecasting.

Time Series Patterns

Time series: A sequence of observations on a variable measured at successive points in time or over successive periods of time.

The measurements may be taken every hour, day, week, month, year, or any other regular interval. The pattern of the data is important in understanding the series—past behavior.

If the behavior of the times series data of the past is expected to continue in the future, it can be used as a guide in selecting an appropriate forecasting method.

- Horizontal Pattern
- Trend Pattern
- Seasonal Pattern
- Cyclical Pattern

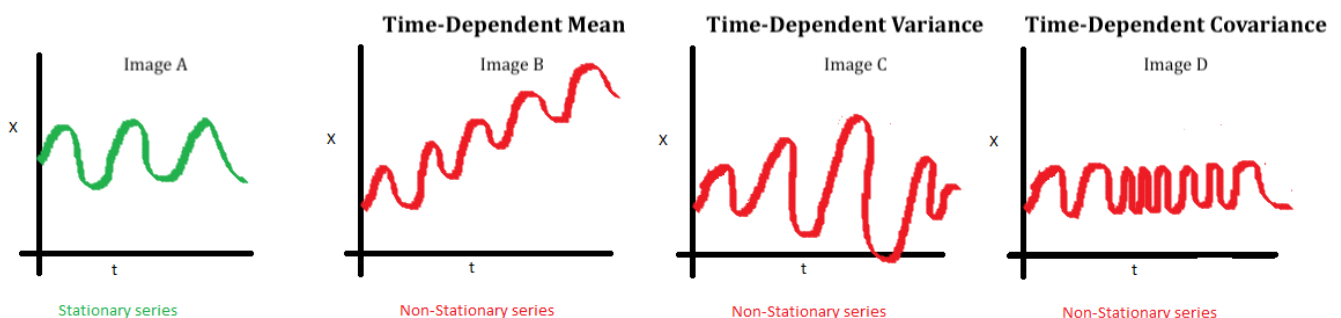
Time series often include multiple patterns.

Time Series Patterns: Horizontal Pattern

Horizontal Pattern exists when the data fluctuate randomly around a constant mean over time.

Stationary time series: It denotes a time series whose statistical properties are independent of time:

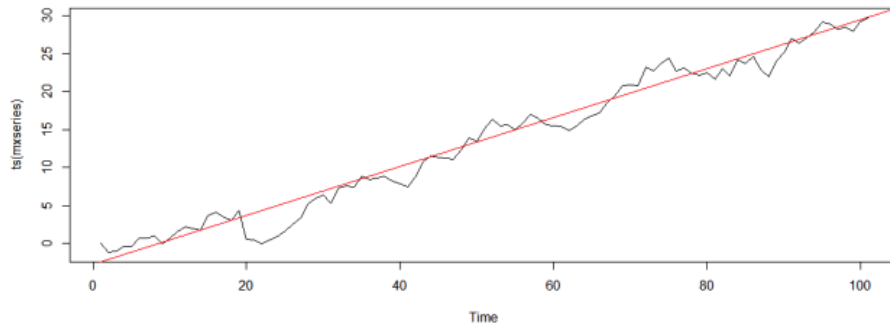
- The process generating the data has a constant mean.
- The variability of the time series is constant over time.



A time series plot for a stationary time series will always exhibit a horizontal pattern with random fluctuations.

Time Series Patterns: Trend Pattern

A trend pattern shows gradual shifts or movements to relatively higher or lower values over a longer period of time.



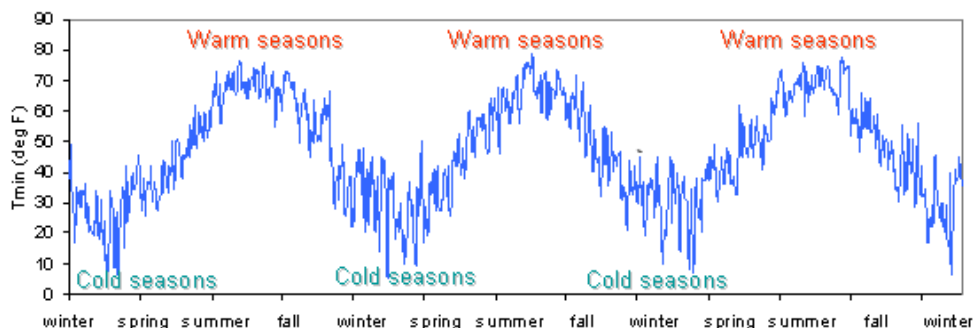
A trend is usually the result of long-term factors such as:

- Population increases or decreases.
- Shifting demographic characteristics of the population.
- Improving technology.
- Changes in the competitive landscape.
- Changes in consumer preferences.

Time Series Patterns: Seasonal Pattern

Seasonal patterns are recurring patterns over successive periods of time.

Example: A retailer that sells bathing suits expects low sales activity in the fall and winter months, with peak sales in the spring and summer months to occur every year.



The time series plot not only exhibits a seasonal pattern over a one-year period but also for less than one year in duration.

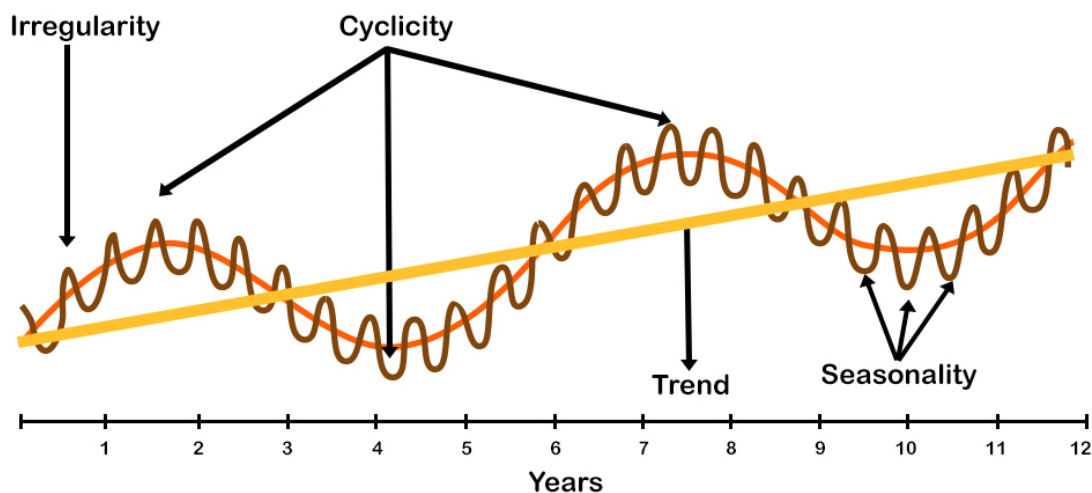
Example: daily traffic volume shows within-the-day “seasonal” behavior, with peak levels occurring during rush hour, moderate flow during the rest of the day, and light flow from midnight to early morning.

Time Series Patterns: Cyclical Pattern

A **cyclical pattern** exists if the time series plot shows an alternating sequence of points below and above the trendline that lasts for more than one year.

Example: Periods of moderate inflation followed by periods of rapid inflation can lead to a time series that alternates below and above a generally increasing trendline.

Cyclical effects are often combined with long-term trend effects and referred to as trend-cycle effects.



Identify and Decompose Time Series Patterns

■ Multiplicative

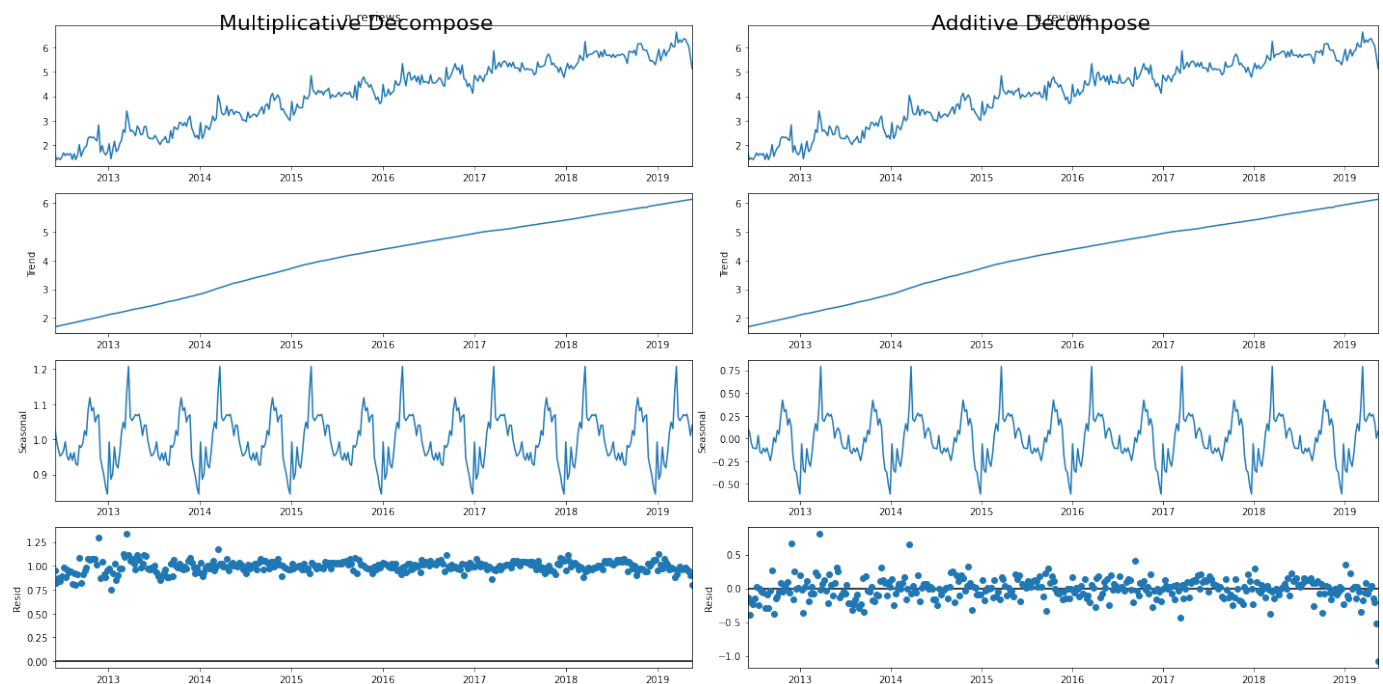
$$TS = Trend \times Seasonality \times Randomness$$

■ Additive

$$TS = Trend + Seasonality + Randomness$$

```
1 from statsmodels.tsa.seasonal import seasonal_decompose
2 # Multiplicative Decomposition
3 result_mul = seasonal_decompose(df_w['n_reviews'] + 0.1, model= '
    multiplicative', extrapolate_trend= 'freq')
4 #Setting extrapolate_trend='freq' takes care of any missing values
    in the trend and residuals at the beginning of the series.
5 # Additive Decomposition
6 result_add = seasonal_decompose(df_w['n_reviews'], model= 'additive
    ', extrapolate_trend='freq')
7 # Plot
8 plt.rcParams.update({'figure.figsize': (10,10)})
9 result_mul.plot().suptitle('Multiplicative Decompose', fontsize=22)
10 result_add.plot().suptitle('Additive Decompose', fontsize=22)
11 plt.show()
```

Visualize Decomposition



Emily Zhu, PhD.

QMST5336: ANALYTICS

© 2023 Texas State University

15 / 29

Using Regression Analysis for Forecasting Trend

Linear Trend Projection:

Regression analysis can be used to forecast a time series with a linear trend. Simple linear regression analysis yields the linear relationship between the explanatory variable and the dependent variable that minimizes the MSE. Use this approach to find a best-fitting line to a set of data that exhibits a linear trend.

- The variable to be forecasted (y_t , the actual value of the time series period t) is the dependent variable
- Trend variable (time period t) is the explanatory variable.

$$\hat{y}_t = b_0 + b_1 t$$

Trend equation for the bicycle sales time series: $\hat{y}_t = 20.4 + 1.1t$

When $t = 11$, $\hat{y}_{11} = 20.4 + 1.1(11) = 32.5$. Thus, the linear trend model yields a sales forecast of 32,500 bicycles for the next year.

We can also use more complex regression models to fit nonlinear trends:

$$\hat{y}_t = b_0 + b_1 t + b_2 t^2 + b_3 t^3$$

Emily Zhu, PhD.

QMST5336: ANALYTICS

© 2023 Texas State University

16 / 29

Using Regression Analysis for Forecasting Seasonality

Seasonality without Trend:

We can model a time series with a seasonal pattern by treating the season as a dummy variable.

$k - 1$ dummy variables are required to model a categorical variable that has k categories

Thus, to model the seasonal effects in the bathing suit sales time series, we need $4-1=3$ dummy variables.

$$\hat{y}_t = b_0 + b_1Qtr1_t + b_2Qtr2_t + b_3Qtr3_t$$

$Qtr1_t = 1$ if period t is quarter 1; 0 otherwise.

$Qtr2_t = 1$ if period t is quarter 2; 0 otherwise.

$Qtr3_t = 1$ if period t is quarter 3; 0 otherwise.

Using Regression Analysis for Forecasting Seasonality C'td

Bathing suits Sales Time Series with Dummy Variables

Period	Year	Quarter	Qtr1	Qtr2	Qtr3	Sales
1	1	1	1	0	0	125
2		2	0	1	0	153
3		3	0	0	1	106
4		4	0	0	0	88
5	2	1	1	0	0	118
6		2	0	1	0	161
7		3	0	0	1	133
8		4	0	0	0	102
9	3	1	1	0	0	138
10		2	0	1	0	144
11		3	0	0	1	113
12		4	0	0	0	80
13	4	1	1	0	0	109
14		2	0	1	0	137
15		3	0	0	1	125
16		4	0	0	0	109
17	5	1	1	0	0	130
18		2	0	1	0	165
19		3	0	0	1	128
20		4	0	0	0	96

Using Regression Analysis for Forecasting Seasonality with Trend

The time series contains both seasonal effects and a linear trend.

$$\hat{y}_t = b_0 + b_1Qtr1 + b_2Qtr2 + b_3Qtr3 + b_4t$$

Quarter 1: Sales = $b_0 + b_1 + b_4t$.

Quarter 2: Sales = $b_0 + b_2 + b_4t$.

Quarter 3: Sales = $b_0 + b_3 + b_4t$.

Quarter 4: Sales = $b_0 + b_4t$.

Autoregressive (AR) models:

Regression models in which the explanatory variables are previous values of the time series.

$$\hat{y}_t = b_0 + b_1y_{t-1} + b_2y_{t-2} + \cdots + b_py_{t-p}$$

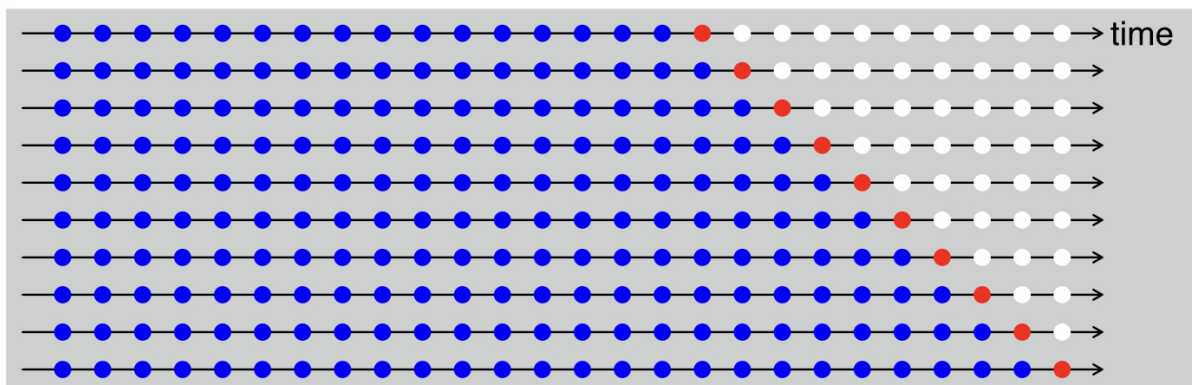
```
1 from statsmodels.tsa.ar_model import AutoReg, ar_select_order
2 #capable of choosing optimal lags using ML algorithms
```

Time Series Forecast: Cross Validation

Traditional evaluation



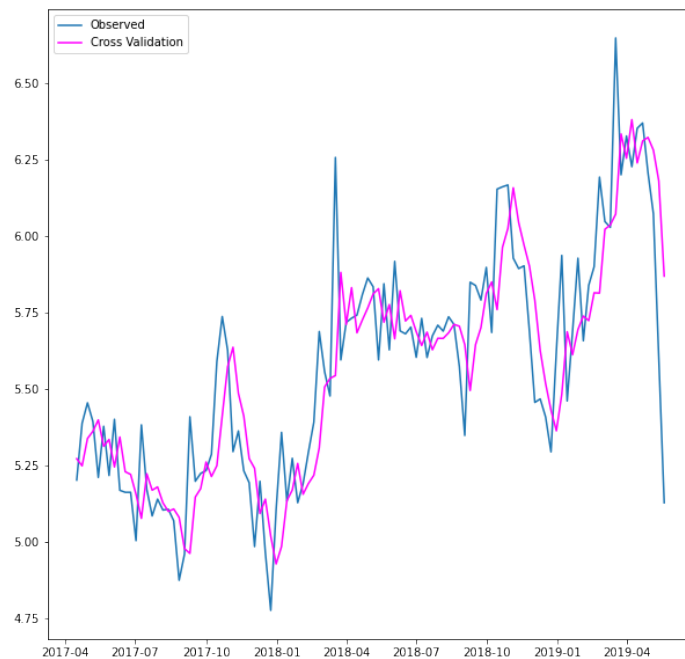
Time series cross-validation



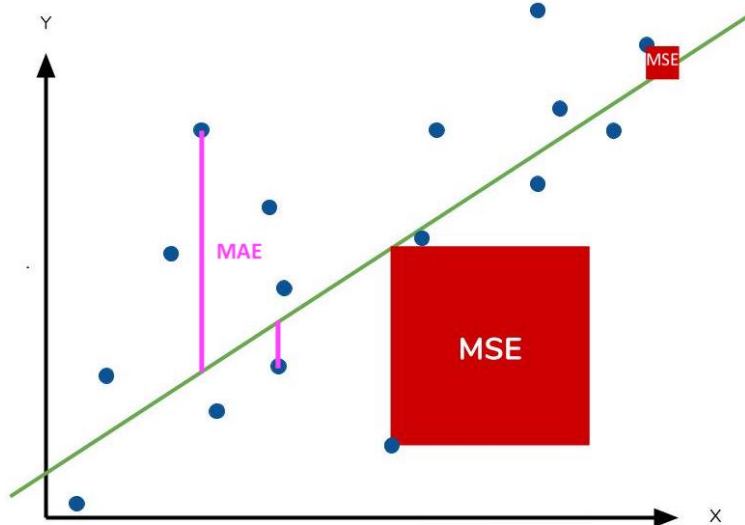
Cross Validation in Python

```
1 series = df_w['n_reviews']
2
3 historic = series.iloc[:int(len(series) * 0.7)] #in-sample/training
4 test = series.iloc[int(len(series) * 0.7):] #out-of-sample/testing
5
6 historic = historic.to_list()
7 predictions = []
8 for i in range(len(test)):
9     sel = ar_select_order(historic, 13, old_names=False)
10    sel.ar_lags
11    model_fit = sel.model.fit()
12    pred = model_fit.predict(start=len(historic), end=len(historic)
13    , dynamic=False)
14    predictions.append(pred[0])
15    historic.append(test[i])
16 cross_val = pd.Series(predictions, index=test.index)
```

Results: Cross Validation



Forecast Accuracy



Measures of Prediction Errors

- MAE: mean absolute errors
- MSE: mean squared errors
 - better than MAE
 - penalize big errors
- RMSE: root mean squared errors
 - better than RMAE
 - penalize big errors
 - the same unit as the dependent variable
- MAPE: mean absolute percentage error
 - average of the absolute value of percentage forecast errors.
 - scaleless

Measures in Python

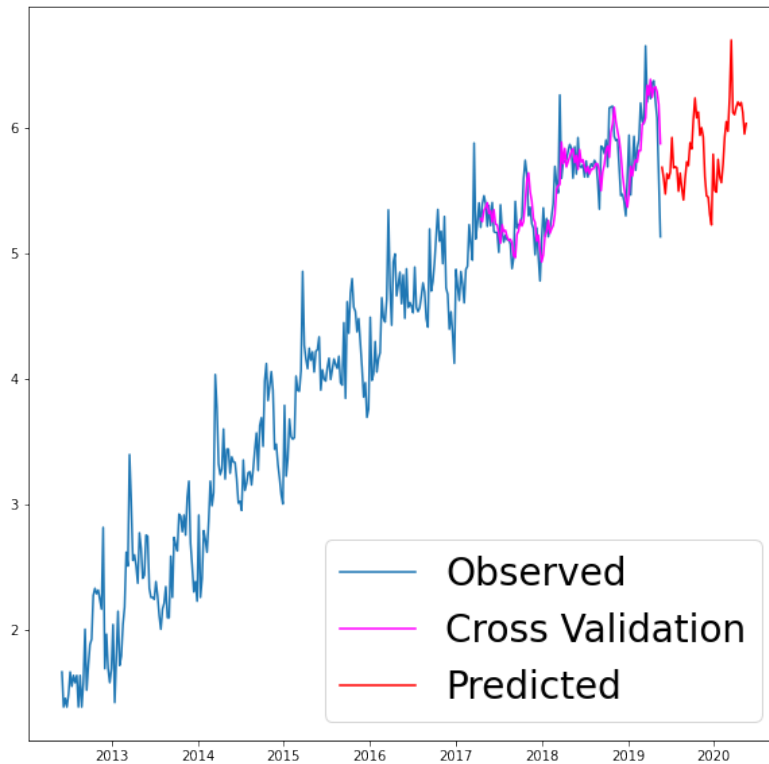
```
1 ### Scikit Learn###
2 from sklearn import metrics
3 # define true and predicted response values
4 y_true = [100, 50, 30, 20]
5 y_pred = [90, 50, 50, 30]
6
7 # calculate MAE, MSE, RMSE and MAPE
8 print(metrics.mean_absolute_error(y_true, y_pred))
9 print(metrics.mean_squared_error(y_true, y_pred))
10 print(metrics.mean_squared_error(y_true, y_pred, squared = False))
11 print(metrics.mean_absolute_percentage_error(y_true, y_pred))

10.0
150.0
12.24744871391589
0.21388334736756193
```

Forecast the Future in Python

```
1 #existing data df_w['n_reviews']
2 historic = series
3 #prediction for the next 52 weeks (1 year)
4 n_pred = 52
5 #the future weeks starts on 2019-05-26, because the last existing
   data ends on 2019-05-19
6 date_pred = pd.date_range("2019-05-26", periods=n_pred, freq="w")
7
8 sel = ar_select_order(historic, 13, glob = True, seasonal = True,
   old_names=False)
9 sel.ar_lags
10 model_fit = sel.model.fit()
11 pred = model_fit.predict(start=len(historic), end=len(historic) +
   n_pred - 1, dynamic=False)
12
13 predictions = pd.Series(pred.values, index=date_pred)
14 # plot results
15 plt.plot(series, label='Observed')
16 plt.plot(cross_val, color='magenta', label='Cross Validation')
17 plt.plot(predictions, color='red', label='Predicted')
18 plt.legend(fontsize = 28)
19 plt.show()
```

Results: Forecast the Future in Python



Statistical Models V.S. Machine Learning

Statistical Models

- sample & population
- hypothesis testig
- confidence interval
- proper model: why & how models work

Machine Learning

- training/testing sets
- MAE, MSE, RMSE, MAPE
- emphasis on predicting
- models can be “black box”

Simple methods can be powerful. We should be cautious about the “black box”.



Next Week

- **Assignment 2** due on October 11.
- Attendance required for group meeting on October 12.

